Project Vulner

Cybersecurity Vulnerability Detection Platform

Project Progress Report — April 2026

1. Project Overview

Project Vulner is an enterprise-grade SaaS cybersecurity platform providing **automated vulnerability detection** through intelligent code analysis. Developers and security teams submit raw source code or entire Git repository URLs; the platform processes submissions asynchronously and returns comprehensive vulnerability assessments enriched with AI-based confidence metrics. The system supports scanning across multiple languages including C/C++, Python, JavaScript, Java, C#, Go, Ruby, and PHP.

Core Technology Stack

- **Frontend:** Next.js 16 / React 19 with Tailwind CSS, Framer Motion, Three.js
- **Backend:** ASP.NET Core 9 Web API — 4-layer DDD (Domain, Application, Infrastructure, API)
- **AI Engine:** Python FastAPI microservice for vulnerability classification
- **Data & Jobs:** SQL Server with EF Core; Hangfire for persistent background processing

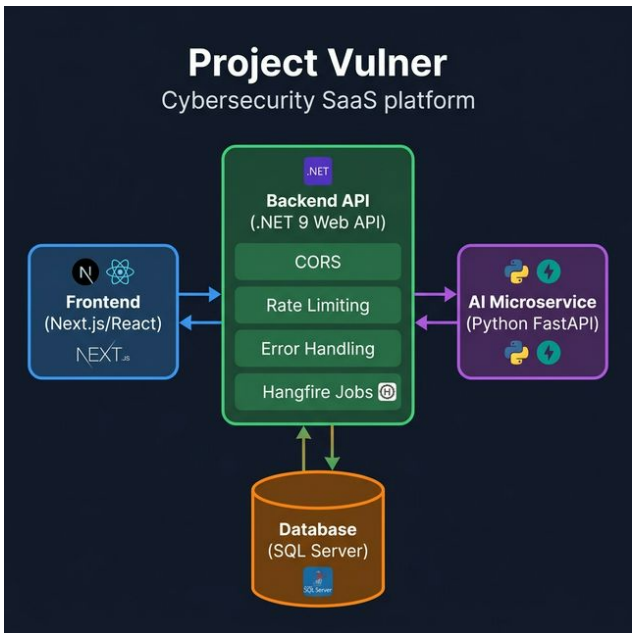


Figure 1: Three-tier system architecture showing the Next.js frontend, .NET 9 backend with middleware pipeline, Python FastAPI AI service, and SQL Server persistence layer.

2. Work Completed

2.1 Backend API and Database

The server-side foundation uses ASP.NET Core 9 partitioned into four architectural layers enforcing strict separation of concerns. The `CodeScan` entity tracks scan ID (GUID), type (Code/RepoUrl), submitted payload, branch, status lifecycle (*Pending* → *Running* → *Completed* | *Failed*), vulnerability flag, AI confidence score, raw JSON response, and creation timestamp.

2.2 Background Processing with Hangfire

Repository cloning and file-by-file AI inference are delegated to **Hangfire** background jobs backed by SQL Server. The `CodeScanJob` orchestrates Git clone operations via LibGit2Sharp, extracts source files by extension (with 1 MB per-file and 100 MB per-repo caps), dispatches each to the AI endpoint, and aggregates results.

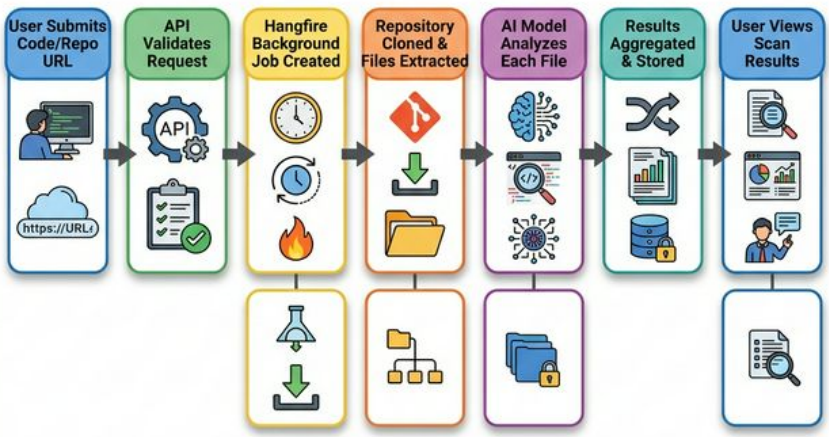


Figure 2: End-to-end scanning workflow: user submission, validation, background job, repository cloning, AI analysis, result aggregation, and final reporting.

2.3 AI Microservice and Security Middleware

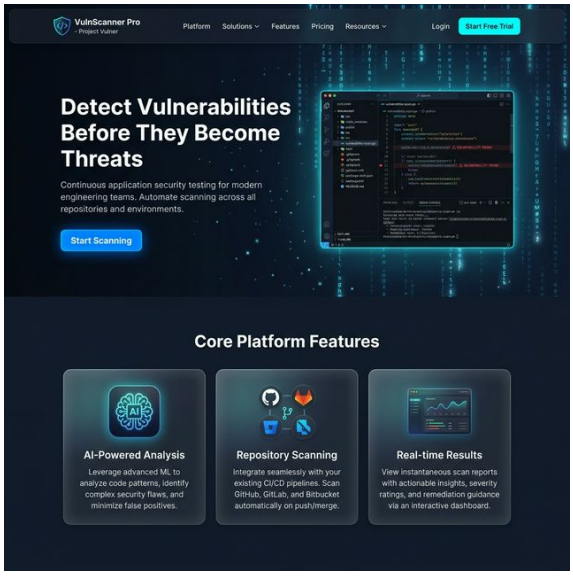
An isolated Python FastAPI service exposes `POST /scan` for vulnerability classification. The .NET `AiScanner` client uses `HttpClientFactory` with Polly policies (3-retry exponential backoff, 60s timeout).

Implemented Security Layers

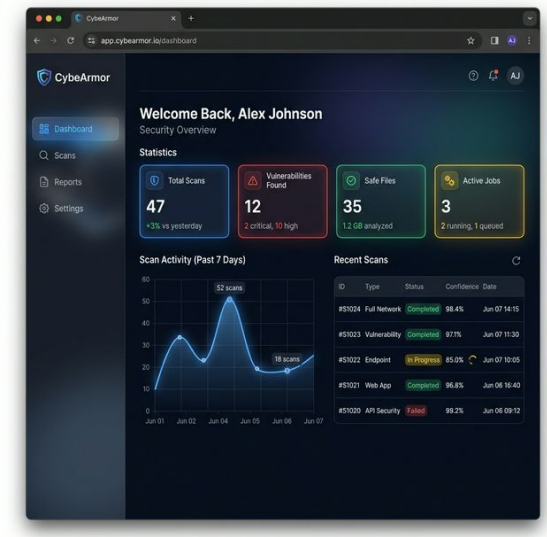
1. **CORS Policy** — Whitelisted frontend origins with explicit header/method grants
2. **Global Error Handler** — Converts exceptions to consistent JSON responses with trace IDs
3. **Rate Limiting** — 10 req/min and 100 req/hour per IP (HTTP 429 response)
4. **Input Validation** — Max 100,000 characters for code; 100 MB repository cap; 1 MB file cap

2.4 Frontend Interface

The client-side application was built with Next.js 16 and React 19, featuring a cybersecurity-themed landing page with particle effects, animated 3D elements via Three.js, and glassmorphism card designs.



(a) Landing page with cyber-themed hero



(b) Dashboard with scan analytics

Figure 3: Project Vulner web application: (a) animated landing page and (b) admin dashboard with statistics tab.

3. Challenges Faced

Challenge 1: Cross-Origin Network Failures

Frontend requests were rejected with 403 errors due to missing CORS headers. **Fix:** Explicit CORS profiles in `Program.cs` whitelisting frontend origins; AI service configured for backend-only access.

Challenge 2: Repository Timeout Cascades

Large repos caused cumulative timeouts during sequential AI inference calls. **Fix:** Granular per-file operations with 60s individual timeouts, repository-level size caps, and Polly retry policies to absorb transient failures.

Challenge 3: Inconsistent AI Output Structure

Unpredictable JSON structures from the AI model caused deserialization crashes. **Fix:** A generic `ServiceResult<T>` wrapper guarantees uniform response envelopes; strict TypeScript interfaces and C# records ensure end-to-end type safety.

4. Work in Progress

Two parallel development tracks are currently active:

1. **Differential Scanning** — Expanding `ICodeScanService` to skip unmodified files based on Git commit hashes, reducing redundant AI inference calls.
2. **Dashboard Components** — Building `AllScansTable` (sortable/filterable), `ScanFilters` (severity filtering), and `ScanDetails` modal panels with per-file drilldown views.

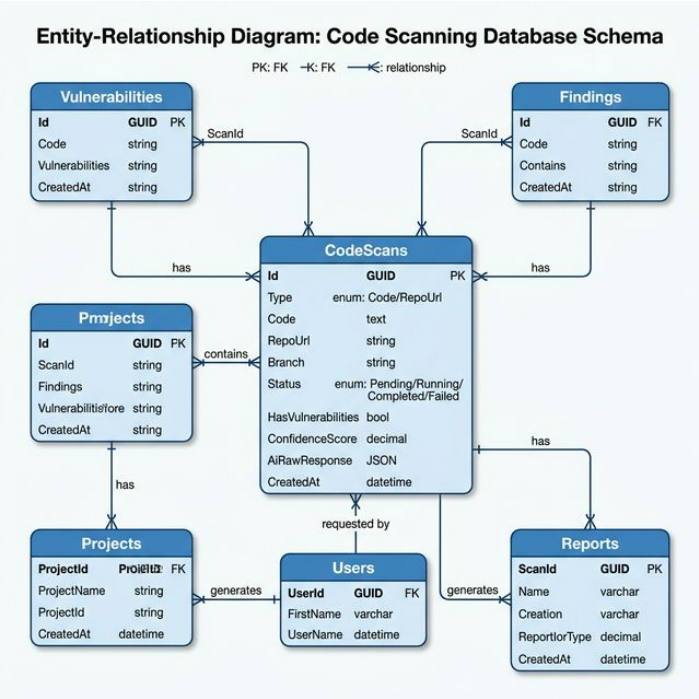


Figure 4: Entity-Relationship diagram of the code scanning database centered on the `CodeScans` table.

5. Next Steps

Priority	Task	Phase
High	JWT authentication with RBAC (Admin/User)	II
High	Cursor-based pagination for scan endpoints	II
Medium	Docker containerization via `docker-compose`	II
Medium	CI/CD pipeline with GitHub Actions	III
Low	Comprehensive unit and integration test suites	III

Table 1: Prioritized development roadmap for upcoming project phases.

6. Team Contribution

Role	Responsibilities
Backend Architect	.NET 4-layer architecture, EF Core models, Hangfire pipelines, middleware orchestration
AI Engineer	FastAPI inference engine, ML model training, inter-service communication protocol
Frontend Developer	Next.js application, Axios client setup, responsive UI/UX, 3D animations
QA & Security	CORS testing, rate-limit stress testing, input validation, integration checklist

Table 2: Team responsibility matrix.

All members participate in cross-functional code reviews and collaborative debugging sessions to maintain holistic system understanding.